Repository-Scale Adaptive Graph Visualization for Codebase Comprehension

Vanvi Rao, Dhruvika Varshney, Ananya Shrivastava
Department of Computer Science and Engineering
Vellore Institute of Technology, Chennai, India
Project Guide: Dr. Sudheer Kumar E.

Abstract

Understanding an unfamiliar software repository requires developers to construct a mental model of its structure, dependencies, and interactions. As repositories increase in size and complexity, manually tracing relationships between files and code components becomes increasingly difficult. This work presents a repository-scale interactive visualization approach for supporting codebase comprehension through dependency graphs. The proposed system retrieves public GitHub repositories and applies AST-based static analysis to Python source code to identify files, imports, classes, functions, function calls, and their relationships. These relationships are represented as an interactive graph that enables users to explore the structure of a codebase visually. The study further investigates whether repository size can be used as a criterion for automatically selecting an appropriate graph layout. A controlled user study is proposed to compare fixed force-directed and hierarchical layouts with a size-adaptive layout across small and large repositories. Code-comprehension tasks will be evaluated using accuracy, task completion time, and perceived effort. The current implementation provides a working prototype for repository retrieval, analysis, dependency extraction, and interactive visualization, establishing the foundation for evaluating adaptive graph layouts for software-architecture comprehension.

Keywords: Repository Visualization, Software Visualization, Graph Layout, Code Comprehension, Static Analysis, Abstract Syntax Tree, Adaptive Visualization

1. Introduction

Understanding an unfamiliar software repository requires developers to construct a mental model of its structure, dependencies, components, and interactions. This process becomes increasingly challenging as software repositories grow in size and complexity, since manually tracing relationships between files and modules can require substantial effort. The increasing scale of modern codebases therefore creates a need for tools that can present software structure in a form that is easier to explore and comprehend.

Software visualization provides one approach to addressing this challenge by representing relationships within a codebase visually. In particular, dependency graphs can provide an overview of how files and code components are connected, allowing developers to explore structural relationships that may be difficult to identify through source-code inspection alone. However, the effectiveness of a graph visualization can depend on how the underlying graph is organized and presented.

Existing research has examined software visualization, graph layout techniques, and code-comprehension tasks, but the choice of graph layout is generally fixed or left to the user. Previous work also suggests that different layouts can be more suitable for different visualization tasks. This raises the question of whether characteristics of the repository itself, particularly its size, can be used to determine an appropriate visualization layout automatically. The literature reviewed for this study did not identify a direct evaluation of repository size as a criterion for automatically selecting graph layouts for software-architecture comprehension.

This work presents a repository-scale interactive graph visualization system designed to support codebase comprehension. The current implementation retrieves public GitHub repositories and performs AST-based static analysis of Python source code to extract files, imports, classes, functions, function calls, and their relationships. These relationships are converted into an interactive dependency graph that allows users to explore the structure of a repository visually.

Building on this system, the study investigates whether a size-adaptive graph layout can improve code comprehension compared with fixed layouts. The proposed evaluation considers small and large repositories and compares

force-directed and hierarchical layouts with a size-adaptive approach. Comprehension will be evaluated using task accuracy, completion time, and perceived effort. Through this approach, the work aims to examine whether repository scale can serve as a useful criterion for adaptive visualization and contribute to more effective tools for understanding unfamiliar software systems.

2. Related Work

2.1 AI-Generated Repository Summarization

Recent research has explored the use of artificial intelligence for generating summaries of software repositories at scales larger than individual functions. Oskooei et al. [1] proposed a hierarchical summarization pipeline that constructs an abstract representation of a repository's file structure and provides project-level context to a large language model before generating summaries at the project, directory, and file levels. This top-down approach is intended to address the limited context available to large language models and was evaluated on bug-localization tasks using industrial data.

Other approaches have investigated structured and module-level documentation generation. HGEN [2] incrementally converts source code into structured documentation while maintaining links between generated documents and their corresponding files. HMCS [3], in contrast, focuses on module-level summaries and argues that this level of abstraction can provide a faster path toward understanding the overall architecture of a software system.

Code-Craft [4] incorporates the structural relationships within a codebase by constructing a hierarchical graph representation to improve context retrieval for code summarization. Similarly, GraphCoder [5] uses a code-context graph to guide repository-level code completion. Agent4cs [6] extends this direction through multiple cooperating language-model agents for summarizing large, hierarchically organized codebases.

Although these approaches demonstrate the potential of repository-level summarization, their evaluation primarily focuses on the quality of the generated text, including coherence, factual accuracy, and retrieval performance. The reviewed work does not directly evaluate whether these representations help human developers construct a mental model of a

software system more efficiently. Furthermore, these approaches are not combined with an interactive visual representation of the repository or evaluated through controlled human comprehension studies.

2.2 Graph Layout Algorithms for Software Structure

Graph visualization research has investigated how different layout algorithms affect the interpretation of software structures and other complex graphs. Di Bartolomeo et al. [7] conducted a systematic review of graph-layout evaluation methods and found that existing evaluation practices are often inconsistent, task-agnostic, or adapted from unrelated application domains. This suggests that there is no universally optimal graph layout and that layout effectiveness should be evaluated in relation to the specific task and structure being visualized. Literature_Review (1)

Devkota et al. [8] addressed this issue by developing layout support specifically around the tasks performed by developers when working with control-flow graphs. Related work on reading control-flow graphs [9] has similarly explored layouts that reflect the visual structure of source code, based on the idea that developers' existing familiarity with source-code patterns may reduce the cognitive effort required to interpret a graph. Literature_Review (1)

Interactive visualization has also been explored as a means of making software-structure diagrams more navigable and understandable. Work presented at VISSOFT 2024 [10] focused on interactive diagrams for software documentation, providing an approach that is architecturally relevant to interactive repository visualization, although it does not specifically address repository-scale dependency graphs. Literature_Review (1)

Despite this body of work, repository size has not been systematically examined as a factor in graph-layout selection. Existing studies generally compare different layouts with respect to visualization tasks or graph structures, rather than investigating how layout effectiveness changes as the scale of the input repository increases. There is also limited work on automatically selecting or adapting layouts as repository scale changes.

2.3 Visualization and AI-Assisted Code Explanation

A more recent direction combines interactive visualization with AI-generated explanations to support developers in understanding unfamiliar software. CodeVoyager [11], for example, combines an interactive call-graph visualization with large-language-model-generated explanations. A comparative user study involving sixteen participants reported lower perceived cognitive effort for users of the combined visual-and-explanation interface than for users of a conventional chat-based code assistant.

However, this work uses another AI-based tool as its comparison baseline and does not investigate repository size or graph-layout choice as independent variables. Consequently, it does not determine whether the effectiveness of an interactive visualization changes with repository scale or whether automatically adapting the graph layout can improve code comprehension.
Graph visualization research has investigated how different layout algorithms affect the interpretation of software structures and other complex graphs. Di Bartolomeo et al. [7] conducted a systematic review of graph-layout evaluation methods and found that existing evaluation practices are often inconsistent, task-agnostic, or adapted from unrelated application domains. This suggests that there is no universally optimal graph layout and that layout effectiveness should be evaluated in relation to the specific task and structure being visualized.

Devkota et al. [8] addressed this issue by developing layout support specifically around the tasks performed by developers when working with control-flow graphs. Related work on reading control-flow graphs [9] has similarly explored layouts that reflect the visual structure of source code, based on the idea that developers' existing familiarity with source-code patterns may reduce the cognitive effort required to interpret a graph.

Interactive visualization has also been explored as a means of making software-structure diagrams more navigable and understandable. Work presented at VISSOFT 2024 [10] focused on interactive diagrams for software documentation, providing an approach that is architecturally relevant

to interactive repository visualization, although it does not specifically address repository-scale dependency graphs.

Despite this body of work, repository size has not been systematically examined as a factor in graph-layout selection. Existing studies generally compare different layouts with respect to visualization tasks or graph structures, rather than investigating how layout effectiveness changes as the scale of the input repository increases. There is also limited work on automatically selecting or adapting layouts as repository scale changes.

Research Direction	Representative Work	How the problem is addressed	Implication for our project	Metrics used
Static code/dependency visualization	Helveg; Semantic Code Graph	Extract code structure and dependencies and represent them as graphs.	Use static analysis as the core technical pipeline; do not claim graph generation itself as novel.	Structural information and dependency relationships; no specific user-performance metric stated
Interactive / multi-level visualization	Helveg; ChangePrism; CodeMap	Provide overview views, filtering, drill-down and multiple abstraction levels.	Provide overview-to-detail exploration and keep graph nodes connected to source information.	Navigation, filtering, drill-down, and exploration effectiveness
Static + dynamic analysis	Krause-Glau et al.	Combine source structure with	Keep dynamic analysis	Runtime behavior and Git workflow

		runtime behavior and Git workflow information.	outside the MVP; it is unnecessary for the core comprehension question.	information
Layout evaluation	Sharif et al.; Di Bartolomeo et al.	Compare layouts using controlled tasks and technical/human-centered metrics.	Use controlled comprehension tasks and report accuracy, time and effort; technical graph metrics can be supplementary.	**Accuracy, task completion time, perceived effort**; technical graph metrics may also be used
User-centered design	Heidrich & Schreiber; CodeMap	Iteratively design and evaluate visualizations with target users.	Include a user study rather than evaluating only the visual output.	User feedback, usability, and visualization evaluation
AI-assisted explanation	CodeMap; Comprendia	Use LLMs to explain code or provide context grounded in structural information.	AI explanation should be a supporting feature, not the claimed core novelty.	Quality and usefulness of generated explanations; contextual relevance
Adaptive visualization	Yanez et al.; recent adaptive/layout research	Change visualization based on user or	Investigate whether repository size can act	Visualization effectiveness based on user/graph

		graph-related information.	as an explicit adaptation signal for software architecture visualization.	conditions; task performance can be evaluated

2.4 Research Gap

The reviewed literature establishes three relevant research directions. First, AI-based approaches can summarize large codebases and provide structured representations of repository information, but their effectiveness is primarily evaluated in terms of generated-text quality rather than measurable improvements in human comprehension. Second, graph-layout research demonstrates that layout choice can influence how users interpret visual structures, but repository scale has rarely been treated as a factor in layout evaluation. Third, interactive visualization combined with AI-generated explanations has shown potential for reducing perceived effort during code understanding, but existing work has not directly examined layout adaptation across repositories of different sizes.

Based on these observations, the specific research gap addressed by this work is the absence of a direct evaluation of whether graph layouts can be automatically adapted according to repository size to improve software-architecture comprehension. Rather than fixing a single layout or requiring users to select one manually, this work investigates whether repository characteristics can guide layout selection.

To investigate this gap, the proposed study compares fixed force-directed and hierarchical layouts with a size-adaptive layout across small and large repositories. The evaluation is based on code-comprehension tasks, with accuracy, completion time, and perceived effort used as measures of effectiveness. AI-generated node-level summaries are intended to be held

constant across the experimental conditions so that layout adaptivity remains the primary variable under investigation.

3. Research Question and Objectives

3.1 Research Question

The primary research question investigated in this work is:

RQ1: Can a size-adaptive graph layout improve code comprehension over fixed layouts?

3.2 Objectives

The objectives of this work are:

1. To develop a web-based tool for interactive repository visualization. 2. To analyze Python repositories using AST-based static analysis. 3. To extract files, imports, classes, functions, and relevant relationships from source code. 4. To construct an interactive dependency graph representing relationships between code components.
5. To compare hierarchical and force-directed graph layouts. 6. To evaluate layout effectiveness using code-comprehension tasks. 7. To measure comprehension accuracy, task completion time, and perceived effort.
8. To investigate whether repository size can support adaptive graph-layout selection.

4. Proposed Methodology

4.1 Repository Selection

The proposed study will evaluate repository visualization across repositories of different sizes in order to investigate whether repository scale can influence the effectiveness of graph-layout selection. Both small and large public Python repositories will be considered. The repositories will be processed using the same analysis pipeline so that differences in visualization

and comprehension can be examined while maintaining a consistent analysis procedure.

4.2 Repository Analysis and Dependency Extraction

Each selected repository will be retrieved from GitHub and analyzed using AST-based static analysis. The analysis will identify relevant structural elements of the source code, including files, imports, classes, functions, and function calls. Relationships between these components will then be extracted to represent the dependencies and interactions present within the repository. This process provides the structural information required to construct the repository-level dependency graph.

4.3 Dependency Graph Construction

The extracted repository information will be represented as a graph in which code components are represented as nodes and their relationships are represented as edges. The resulting dependency graph will provide an interactive representation of the repository structure. Users will be able to explore relationships between code components and inspect relevant information associated with the visualized elements.

4.4 Graph Layouts

Two fixed graph-layout approaches, force-directed and hierarchical layouts, will be considered in the proposed evaluation. These layouts will be applied to repositories of different sizes and compared using code-comprehension tasks. The purpose of this comparison is to examine whether the effectiveness of a layout changes with repository scale and whether a fixed layout is equally suitable across different repository sizes.

4.5 Size-Adaptive Layout

Based on the comparison of layouts across repository sizes, a size-adaptive layout approach will be investigated. The proposed approach will use repository characteristics, particularly repository size, as a basis for selecting an appropriate graph layout. The adaptive approach will then be compared with the fixed-layout conditions to determine whether automatically adapting

the visualization can improve code comprehension**.**

4.6 User study

A controlled user study involving approximately 15–20 participants is proposed to evaluate the effectiveness of the different visualization approaches. Participants will be presented with repository visualizations and asked to complete code-comprehension tasks. The study will compare user performance across the fixed and size-adaptive visualization conditions.

4.7 Evaluation Metrics

The effectiveness of the visualization approaches will be evaluated using three primary measures: comprehension accuracy, task completion time, and perceived effort. Accuracy will indicate how correctly participants complete the comprehension tasks, while completion time will capture the time required to obtain the required information. Perceived effort will provide an indication of the cognitive effort experienced by participants while using the different visualization approaches. The results will be compared across layouts and repository sizes to determine whether the size-adaptive approach provides measurable benefits over fixed layouts.

5. System Architecture

5.1 Repository Retrieval

The system accepts a public GitHub repository as its input. The repository is retrieved and prepared for analysis before the source code is processed. This provides a consistent starting point for extracting structural information from different repositories.

5.2 Source Code Parsing and Static Analysis

After retrieval, the source code is parsed using Abstract Syntax Tree (AST)-based static analysis. The analysis focuses on identifying structural elements within Python repositories, including files, imports, classes, functions, and function calls. Since the analysis is performed statically, the repository structure can be examined without requiring the program to be

executed.

5.3 Dependency Graph Construction

The structural information extracted during analysis is used to construct a dependency graph representing relationships between code components. Files and other relevant code components form the elements of the graph, while their relationships are represented as connections. This transforms the repository from a collection of source files into a visual representation of its structural organization.

5.4 Interactive Visualization

The constructed dependency graph is presented as an interactive visualization. Users can explore the relationships between code components and navigate the repository through its graphical representation. The system also provides information about individual code components, supporting users in understanding the structure of an unfamiliar codebase.

5.5 Overall Architecture

The complete system therefore follows a sequential pipeline in which a public repository is retrieved, parsed, analyzed, and transformed into a dependency graph before being presented through an interactive visualization interface. This architecture provides the foundation for comparing different graph layouts and subsequently evaluating size-adaptive layout selection.

6. Implementation and Technology Stack

6.1 Technology Stack

The proposed system uses a web-based architecture combining a modern frontend with a Python-based backend for repository analysis. The frontend is implemented using React, Vite, and TypeScript, with CSS and Tailwind CSS used for styling and user-interface development. The backend uses Python and FastAPI to handle repository processing and analysis. Python's AST capabilities and custom parsers are used to extract structural information from source code, while GitPython provides access to public GitHub repositories. The graph visualization is implemented using custom SVG-based rendering.

Python is also used for research analysis and evaluation.

Table 1. Proposed Technology Stack

Component	Technology
Frontend	React + Vite
Styling and UI	Tailwind CSS + CSS + Motion
Backend	Python + FastAPI
Code Analysis	Python AST + custom static analysis/parsers
Graph Visualization	Cytoscape.js + Dagre+ FCose
Repository Access	GIt + GitPython+ GitHub
Research Analysis	Python

6.2 Current Implementation

A working prototype of the repository visualization system has been developed. The prototype accepts a public GitHub repository URL as input and automatically retrieves and analyzes the repository. The system extracts structural information including files, classes, functions, imports, and dependencies, and identifies relationships between the extracted code components. These relationships are then converted into an interactive visual representation of the codebase, allowing users to explore the repository structure through the generated graph.

The current implementation establishes the core functionality required for the proposed research evaluation. In particular, it provides the repository retrieval, source-code analysis, dependency extraction, and interactive visualization components needed to generate graph representations of software repositories. The prototype therefore serves as the experimental platform for the subsequent comparison of graph layouts and evaluation of size-adaptive visualization.

7. Results and Initial Implementation

7.1 Prototype Functionality

The initial implementation resulted in a working prototype capable of processing public GitHub repositories and generating interactive visual representations of their code structure. The system accepts a repository URL as input, retrieves the repository, and performs automated source-code analysis. The analysis identifies files, classes, functions, imports, and dependencies within the repository. IDP_Review-2

The extracted structural information is used to identify relationships between code components and construct an interactive dependency graph. The resulting visualization enables users to explore the organization and relationships of components within the analyzed repository, providing a visual representation of the codebase structure.

7.2 Current Capabilities

The current prototype establishes the core functionality required for the proposed visualization study. Its capabilities include repository retrieval, automated source-code analysis, extraction of structural relationships, and interactive visualization of the resulting dependency graph. These capabilities provide the foundation for generating the different graph-layout conditions required for the planned user study.

8. Discussion and Expected Evaluation

The proposed study is designed to examine whether graph-layout effectiveness changes with repository scale and whether adapting the layout to repository characteristics can improve code comprehension. The comparison between fixed force-directed and hierarchical layouts will provide a baseline for evaluating the effect of layout choice across repositories of different sizes.

The size-adaptive approach is expected to provide a more suitable visualization when the structure of a repository becomes more complex. However, the effectiveness of the adaptive approach cannot be assumed in advance and will be determined through the proposed user study.

Comprehension accuracy, task completion time, and perceived effort will be compared across the experimental conditions to identify differences in user performance.

The evaluation will also help determine whether repository size is a useful criterion for layout selection. If the adaptive approach consistently performs better than fixed layouts across the evaluated repositories, the results would provide evidence supporting size-based layout adaptation. Conversely, if no meaningful improvement is observed, the findings would indicate that repository size alone may not be sufficient for selecting an effective graph layout and that additional repository characteristics may need to be considered.

9. Future Enhancements

Several extensions can be considered to further improve the proposed repository visualization system. The interactive dependency graph can be enhanced with additional navigation and inspection features, including file-level node inspection, repository search and filtering, and repository-level metrics. Improved loading states and error handling can also make the analysis workflow more reliable when processing different repositories.

The visualization component can be extended to support multiple graph layouts, including hierarchical, force-directed, and circular layouts. A key future direction is the implementation of size-adaptive layout selection, in which repository characteristics are used to automatically determine an appropriate visualization layout. This functionality can subsequently be evaluated against fixed-layout approaches through controlled user studies.

Another potential enhancement is the integration of AI-generated explanations for code components. Such explanations could provide additional contextual information about individual files, classes, or functions and may further support developers when exploring unfamiliar repositories. This extension can be investigated separately so that the effect of graph-layout adaptation remains distinguishable from the effect of AI-generated explanations.

10. Conclusion

This work presents a repository-scale interactive graph visualization approach for supporting codebase comprehension. The proposed system retrieves public GitHub repositories and uses AST-based static analysis to extract structural information from Python code, including files, imports, classes, functions, and dependencies. The extracted relationships are represented through an interactive dependency graph that allows users to explore the structure of an unfamiliar repository visually.
The central focus of the study is to investigate whether repository size can be used as a criterion for selecting an appropriate graph layout. The proposed evaluation compares fixed force-directed and hierarchical layouts with a size-adaptive approach across repositories of different sizes. Comprehension accuracy, task completion time, and perceived effort are proposed as the primary evaluation measures.

The current prototype establishes the core technical foundation required for this investigation by providing repository retrieval, static analysis, dependency extraction, and interactive visualization. The proposed study will build upon this implementation to determine whether adaptive graph layouts can provide measurable benefits for software-architecture comprehension and whether repository scale can serve as a useful basis for visualization adaptation.

References

[1] A. R. Oskooei, S. Yukcu, M. C. Bozoglan, and M. S. Aktas, “Repository-Level Code Understanding by LLMs via Hierarchical Summarization: Improving Code Search and Bug Localization,” in *Computational Science and Its Applications – ICCSA 2025 Workshops*, pp. 88–105, 2025.

[2] K. R. Dearstyne, A. D. Rodriguez, and J. Cleland-Huang, “Supporting Software Maintenance with Dynamically Generated Document Hierarchies,” arXiv:2408.05829, 2024.

[3] W. Sun, Y. Zhang, J. Zhu, Z. Wang, C. Fang, Y. Zhang, Y. Feng, J. Huang, X. Wang, Z. Jin, and Y. Liu, “Commenting Higher-level Code Unit: Full Code, Reduced Code, or Hierarchical Code Summarization,”

arXiv:2503.10737, 2025.

[4] D. Sounthiraraj, J. Hancock, Y. Kortam, A. Javvaji, P. Singh, and S. Shankar, “Code-Craft: Hierarchical Graph-Based Code Summarization for Enhanced Context Retrieval,” arXiv:2504.08975, 2025.

[5] W. Liu, A. Yu, D. Zan, B. Shen, W. Zhang, H. Zhao, Z. Jin, and Q. Wang, “GraphCoder: Enhancing Repository-Level Code Completion via Coarse-to-fine Retrieval Based on Code Context Graph,” in *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, pp. 570–581, 2024.

[6] Y. Tang, E. Sarikayak, D. Tuncel, J. M. Zhang, and T. A. Runkler, “Agent4cs: A Multi-agent System for Code Summarization in Large Hierarchical Codebases,” arXiv:2607.01425, 2026.

[7] S. Di Bartolomeo, T. Crnovrsanin, D. Saffo, E. Puerta, C. Wilson, and C. Dunne, “Evaluating Graph Layout Algorithms: A Systematic Review of Methods and Best Practices,” *Computer Graphics Forum*, vol. 43, no. 6, Art. no. e15073, 2024.

[8] S. Devkota, M. P. LeGendre, A. Kunen, P. Aschwanden, and K. E. Isaacs, “Domain-Centered Support for Layout, Tasks, and Specification for Control Flow Graph Visualization,” in *2022 IEEE Working Conference on Software Visualization (VISSOFT)*, pp. 40–50, 2022.

[9] P. Schaad, T. Ben-Nun, and T. Hoefler, “VEIL: Reading Control Flow Graphs Like Code,” arXiv:2511.05066, 2025.

[10] A. Štěpánek, D. Kuťák, B. Kozlíková, and J. Byška, “Interactive Diagrams for Software Documentation,” in *2024 IEEE Working Conference on Software Visualization (VISSOFT)*, pp. 12–23, 2024.

[11] Y. Kim, K. Jang, H. Song, M. Shin, and B. Suh, “CodeVoyager: An Interactive Tool for Rapid Code Comprehension through LLM-Assisted Graph Navigation,” in *UIST Adjunct 2025: Proceedings of the 38th Annual ACM Symposium on User Interface Software and Technology*, 2025.

[12] F. Galperin, R. Koschke, and M. Steinbeck, “Visualizing Code Smells: Tables or Code Cities? A Controlled Experiment,” in *2022 IEEE Working*

Conference on Software Visualization (VISSOFT), 2022.